Mathematical Notes

Pablo, Arka

July 21, 2026

Abstract

First draft of the research papers to emerge from this project: Rao–Blackwellization, branching and poker case-study.

Contents

1 Introduction **2**

2 Game-Theoretical Model **2**

3 Self-Play Policy-Gradient Methods in Synchronous Rollouts **5**
3.1 Theoretical Model 5
3.2 Usual Implementation 7
3.3 Rao–Blackwellization of the Private State 9
3.4 Variance-Guided Branching 12

4 Loss Estimator Variance **22**
4.1 Sequential Rao–Blackwellization 22

5 Poker: a Perfect Example **23**

1 Introduction

In the context of end-to-end policy learning, we train a model to learn a parametrized strategy profile π^θ. All policy-gradient methods to train such θ are based on a similar principle. They create a sequence $\theta_0, \dots \theta_k$ of parameters where typically θ_0 is a random seed, and they gradually build the next iterate by:

(i) **Rollout:** Sampling games using the current strategy profiles $(\pi_{\theta_i})_{i\leq k}$.

(ii) **Loss design:** Secretly, unbeknownst to the opponents, some new player with strategy π^θ will randomly replace the acting player. We pick some sort of loss function $\mathcal{L}_{k+1}(\theta)$ which depends on $(\theta_i)_{i\leq k}$, and which evaluates how well the new player π^θ is doing.

(iii) **Gradient Descent**: Using minibatches sampled from the online rollout database, we compute some estimator of the loss $\widehat{\mathcal{L}}_{k+1}(\theta)$ and descend along it.

Trajectories in games typically only get assigned to a computable reward at the end of the round (terminal trajectories), which forces rollout to *sample full runouts*. Typically, each runout is sampled by generating an explicit sequence of full game states (all private and public information is decided).

We introduce here two methods to reduce the variance of the loss estimators $\widehat{\mathcal{L}}_{k+1}(\theta)$:

(i) **Rao–Blackwellization of private state:** games have typically a very wide variance coming from the unavailable private information: the exact same strategy can lead to terrible failure or wonderful success depending on the private state of the opponents. We suggest addressing this by integrating away this variance. We replace private-state sampling in the loss estimator by conditional expectations over the posterior distribution of private states.

(ii) **Variance-guided Branching:** a significant amount of variance in the loss estimator typically also comes from the variety of public events (actions and chance events) which can significantly modify the strength of the acting player's private state. We suggest addressing this focusing the sampling around higher variance spots, by seeing rollouts as a tree instead of a set of sequences.

2 Game-Theoretical Model

To simplify the notation, we focus on **finite Public-action Bayesian Games (PBG)**: all private information is sampled initially, after which every player action and chance outcome is public and play is sequential. Player p therefore acts with information state (x_p, σ), and these games have perfect recall. This restricted extensive-form Bayesian class includes many poker, bidding, and auction models, but not dynamic private observations, hidden actions, or simultaneous moves without further extensions.

Definition 2.1 (Finite public-action Bayesian game)**.** For a finite set E, let

$$\Delta(E) := \left\{ \lambda : E \to [0,1] \;\middle|\; \sum_{e\in E} \lambda(e) = 1 \right\}. \tag{2.1}$$

A finite public-action Bayesian game consists of the following data.

(i) **Tree:** A finite prefix-closed set $\mathcal{S}$ of words, seen as a tree, with empty word $\varnothing \in \mathcal{S}$. Its elements are called public states, or trajectories. Write $\sigma \preccurlyeq \tau$ if σ is a prefix of τ,

and $\sigma \prec \tau$ if $\sigma \preccurlyeq \tau$ and $\sigma \neq \tau$. The terminal trajectories are the maximal elements of $\mathcal{S}$ and are denoted by $\mathcal{T}$. For every nonterminal trajectory $\sigma \in \mathcal{S} \setminus \mathcal{T}$, there is a finite event set $\mathcal{E}_\sigma$ such that the immediate successors of σ are precisely the trajectories σe, with $e \in \mathcal{E}_\sigma$.

(ii) **Owner:** A finite set $\mathcal{P}$ of *players* and an owner map $o : \mathcal{S} \setminus \mathcal{T} \to \mathcal{P} \sqcup \{\mathsf{c}\}$. If $o(\sigma) \in \mathcal{P}$ (resp. $\{\mathsf{c}\}$) is a player node (resp. chance node), we typically denote the elements of $\mathcal{E}_\sigma$ by a (resp. c). For $\sigma \in \mathcal{S}$ and $p \in \mathcal{P}$, we write $|\sigma|_p := \#\{\rho \preccurlyeq \sigma \mid o(\rho) = p\}$ for the number of player nodes in σ owned by p, and $|\sigma|_{\text{player}} := \sum_{p \in \mathcal{P}} |\sigma|_p$ for the total number of player nodes in σ. Thus, when $o(\sigma) = p$, the state σ is counted in $|\sigma|_p$: it is the state immediately before p chooses an outgoing action $a \in \mathcal{E}_\sigma$; that action is the edge from σ to the next state σa. For $\rho \preccurlyeq \sigma$, we write $|\sigma \setminus \rho|_p := |\sigma|_p - |\rho|_p$ and $|\sigma \setminus \rho|_{\text{player}} := |\sigma|_{\text{player}} - |\rho|_{\text{player}}$. For $\tau \in \mathcal{T}$, write $\tau^{\mathsf{pre-c}}$ for the largest prefix ending in no more than one node in $\mathcal{T} \sqcup o^{-1}(\mathsf{c})$. In other words, we have $\tau^{\mathsf{pre-c}} = \sigma a$ for $\sigma a \preccurlyeq \tau$ the last player node $o(\sigma) \in \mathcal{P}$ and decision a: if $\sigma a \prec \tau$, we must have $o(\rho) = \mathsf{c}$ for every $\sigma a \preccurlyeq \rho \prec \tau$. Informally, a pre-terminal state is when all players have acted, and before any final chance events are revealed which may impact rewards (*e.g.* when players are all-in in poker before the board is complete, the pre-terminal state is right after the last "*call*" and before seeing any more board cards).

(iii) **Initial Private State Profile:** For every player $p \in \mathcal{P}$, a finite type set $\mathcal{X}_p$, and a distribution μ on $\mathcal{X} := \prod_{p \in \mathcal{P}} \mathcal{X}_p$. Elements of $\mathcal{X}$ are typically written $x = x_{\mathsf{c}} = (x_p)_{p \in \mathcal{P}}$, with $x_p \in \mathcal{X}_p$.

(iv) **Public-Chance Rules:** For every chance node σ and every initial profile $x \in \mathcal{X}$, a public-chance rule $\chi(\cdot \mid x, \sigma) \in \Delta(\mathcal{E}_\sigma)$.

(v) **Reward:** a function $\mathbf{R} = (R_p)_{p \in \mathcal{P}} : \mathcal{X} \times \mathcal{T} \to \mathbb{R}^{\mathcal{P}}$.

Definition 2.2 (Strategy and Random Trajectories)**.** A strategy profile is the data of $\pi(\cdot \mid x_p, \sigma) \in \Delta(\mathcal{E}_\sigma)$ for $\sigma \in o^{-1}(\mathcal{P})$ and $x_{o(\sigma)} \in \mathcal{X}_{o(\sigma)}$. For $p \in \mathcal{P}$, we write π_p for the strategy of player p, *i.e.* the subcollection of these distributions for $p = o(\sigma)$, and π_{-p} for the strategies of all other players. A strategy profile induces a joint probability measure $\mathbb{P}^\pi$ on the set $\mathcal{X} \times \mathcal{T}$.

For a trajectory $\sigma \in \mathcal{S}$, a player p and a private initial profile $x_p \in \mathcal{X}_p$, define the **reach** as the probability that player p acted according to σ conditional on initial profile x_p and every other event following σ:

$$r_p^\pi(\sigma \mid x_p) := \mathbb{P}^\pi(p \text{ acts as } \sigma \mid x_p, \text{other events follow } \sigma) = \prod_{\substack{\rho a \preccurlyeq \sigma \\ o(\rho) = p}} \pi(a \mid x_p, \rho), \tag{2.2}$$

where the product ranges over the actions of player p in the trajectory σ. Similarly, we define the **draw odds** of a trajectory σ as the probability that the public chance events in σ occurred conditional on the initial profile x:

$$\begin{aligned} r_{\mathsf{c}}(\sigma \mid x) &:= \mathbb{P}^\pi(\text{chance happens as in } \sigma \mid x, \text{players act as } \sigma) \\ &= \prod_{\substack{\rho c \preccurlyeq \sigma \\ o(\rho) = \mathsf{c}}} \chi(c \mid x, \rho), \end{aligned} \tag{2.3}$$

The expected payoff of player p is

$$J_p(\pi) := \mathbb{E}^\pi(R_p) = \mathbb{E}_{X\sim\mu}\left(\sum_{\tau\in\mathcal{T}} R_p(X,\tau) \prod_{q\in\mathcal{P}\cup\{\mathsf{c}\}} r_q(\tau \mid X_q)\right) \tag{2.4}$$

For $\sigma \in o^{-1}(\mathcal{P})$, $p = o(\sigma)$ and $x_p \in \mathcal{X}_p$, we write $V(x_p,\sigma) = \mathbb{E}^\pi(R_p(x,\tau) \mid x_p,\sigma)$. For a further choice of $a \in \mathcal{E}_\sigma$, we write $Q(x_p,\sigma a) = \mathbb{E}^\pi(R_p(x,\tau) \mid x_p,\sigma a)$ and $A(x_p,\sigma a) = Q(x_p,\sigma a) - V(x_p,\sigma)$. Here and throughout, a conditional expectation conditioned on an event of probability zero is defined to be zero.

Remark 2.1 (Evolving private states). One could allow the private state to evolve by assigning $\mathcal{X}_\sigma = \prod_p \mathcal{X}_{\sigma,p}$ to every public state. At a player node $o(\sigma) = p$, for each $x_p \in \mathcal{X}_{\sigma,p}$ let

$$\mathcal{D}_\sigma(x_p) \subseteq \coprod_{a\in\mathcal{E}_\sigma} \{a\} \times \mathcal{X}_{\sigma a,p}$$

be a nonempty set of legal decisions. The strategy would be a transition kernel

$$\pi(\cdot \mid x_p,\sigma) \in \Delta(\mathcal{D}_\sigma(x_p)),$$

so that the player jointly chooses the public action and their next private state. At a chance node, we would similarly take

$$\chi(\cdot \mid x,\sigma) \in \Delta\left(\coprod_{c\in\mathcal{E}_\sigma} \{c\} \times \mathcal{X}_{\sigma c}\right).$$

Writing $\mathbf{r}^\pi(\sigma)(x)$ for unnormalised reach mass, and carrying the nonacting coordinates unchanged, reach is defined inductively by

$$\mathbf{r}^\pi(\varnothing)(x) = \mu(x), \qquad \mathbf{r}^\pi(\sigma e)(x') = \begin{cases} \displaystyle\sum_{x:\, x_{-p}=x'_{-p}} \mathbf{r}^\pi(\sigma)(x)\pi(e,x'_p \mid x_p,\sigma) & o(\sigma) = p, \\ \displaystyle\sum_{x\in\mathcal{X}_\sigma} \mathbf{r}^\pi(\sigma)(x)\chi(e,x' \mid x,\sigma) & o(\sigma) = \mathsf{c}. \end{cases}$$

Thus $\mathbb{P}^\pi(\sigma \preccurlyeq \tau) = \sum_x \mathbf{r}^\pi(\sigma)(x)$, and the posterior at σ is its normalisation. These kernels give a much larger class of games to which the same techniques apply. We keep the formalism below closer to poker, where private states are sampled initially, to simplify the notation.

Notation 2.3 (Stationary Distributions in Tree-Shaped Markov Chains)**.** Whenever we have a tree-shaped Markov chain with a collection of terminal states, we call *stationary distribution* the unique probability distribution that is stable under:

(i) at any non-terminal state, randomly moving down the tree according to the transition probabilities, and

(ii) at any terminal state, jumping back to the root of the tree.

From a game persepective, we see games as endlessly restarting whenever someone won.

Notation 2.4 (Ideal Random Variables)**.** We first sample an ideal player-decision node Σ^π according to

$$\mathbb{P}(\Sigma^\pi = \sigma) = \mathbb{1}_{o(\sigma)\in\mathcal{P}} \frac{\mathbb{P}^\pi(\sigma \preccurlyeq \tau)}{\mathbb{E}^\pi(|\tau|_{\text{player}})}. \tag{2.5}$$

Conditionally on $\Sigma^\pi = \sigma$, we let X^π follow the posterior law

$$\mathbb{P}(X^\pi = x \mid \Sigma^\pi = \sigma) = \frac{\mu(x)\mathbb{P}^\pi(\sigma \preccurlyeq \tau \mid X = x)}{\mathbb{P}^\pi(\sigma \preccurlyeq \tau)}, \tag{2.6}$$

we set $P^\pi = o(\Sigma^\pi)$, and we let $A^\pi \sim \pi(\cdot \mid X^\pi_{P^\pi}, \Sigma^\pi)$. Before observing Σ^π, the joint and marginal laws of X^π and P^π are

$$\mathbb{P}(X^\pi = x, P^\pi = p) = \frac{\mu(x)\mathbb{E}^\pi(|\tau|_p \mid X = x)}{\mathbb{E}^\pi(|\tau|_{\text{player}})}, \tag{2.7}$$

and hence

$$\mathbb{P}(X^\pi = x) = \frac{\mu(x)\mathbb{E}^\pi(|\tau|_{\text{player}} \mid X = x)}{\mathbb{E}^\pi(|\tau|_{\text{player}})}, \qquad \mathbb{P}(P^\pi = p) = \frac{\mathbb{E}^\pi(|\tau|_p)}{\mathbb{E}^\pi(|\tau|_{\text{player}})}. \tag{2.8}$$

When π is clear from the context, we drop the superscript and write X, P, Σ and A.

Recall below how the key theorem of policy gradient theory adapts to this setting.

Theorem 2.1 (Policy Gradient Theorem for Games)

For $p \in \mathcal{P}$, using the ideal random variables of Notation 2.4, we have

$$\nabla_{\pi_p} J_p(\pi) = \mathbb{E}^\pi(|\tau|_p)\nabla_{\pi_p}\mathbb{E}^{\text{sg}[\pi]}(\log(\pi(A \mid X_p, \Sigma))\,\text{sg}[Q(X_p, \Sigma A)] \mid P = p) \tag{2.9}$$

Thus the aggregate profile gradient targeted below is the sum of the unilateral gradients in Equation 2.9; equivalently, it weights each player's conditional gradient by $\mathbb{P}(P = p) = \mathbb{E}^\pi(|\tau|_p)/\mathbb{E}^\pi(|\tau|_{\text{player}})$ under the ideal law.

3 Self-Play Policy-Gradient Methods in Synchronous Rollouts

3.1 Theoretical Model

Assuming a *synchronous rollout*, all main policy-gradient methods to solve games can be modelled by the following choices:

1. A **model** is the choice of a parametrized strategy profile $\theta \mapsto \pi^\theta$. Optionally, it also included the choice of a parametrized value functions $\theta \mapsto V^\theta$ meant to approximate $\mathbb{E}^{\pi^\theta}(R_p(x, \tau) \mid x_p, \sigma \preccurlyeq \tau)$ for all $\sigma \in \mathcal{S}$, $p = o(\sigma)$ and $x_p \in \mathcal{X}_p$.

2. Some **value** estimators

$$\widehat{A}(x_p, \sigma a) \simeq \mathbb{E}^\pi(R_p(x, \tau) \mid x_p, \sigma a) - \mathbb{E}^\pi(R_p(x, \tau) \mid x_p, \sigma) \tag{3.1}$$
$$\widehat{Q}(x_p, \sigma a) \simeq \mathbb{E}^\pi(R_p(x, \tau) \mid x_p, \sigma a) \tag{3.2}$$
$$\widehat{V}(x_p, \sigma) \simeq \mathbb{E}^\pi(R_p(x, \tau) \mid x_p, \sigma) \tag{3.3}$$

   for $\sigma a \in \mathcal{S}$, $p = o(\sigma)$ and $x_p \in \mathcal{X}_p$. Optionally, the reward R_p above can be replaced by a modified reward depending on π and some reference profile ρ (see R-NaD).

3. A **loss function** $\mathcal{L}$ from which we want to estimate

$$l(\theta) = \mathbb{E}\Big(\mathcal{L}\Big((\pi^\theta, V^\theta, \pi, V)(X_P, \Sigma), A, \widehat{A}, \widehat{Q}, \widehat{V}\Big)\Big) \tag{3.4}$$

for Σ, A, P, X as in Notation 2.4 and V all based on a frozen policy π, called the *rollout profile* (typically an older version of π^θ), and for $\widehat{A} = \widehat{A}(X_P, \Sigma A)$, $\widehat{Q} = \widehat{Q}(X_P, \Sigma A)$ and $\widehat{V} = \widehat{V}(X_P, \Sigma)$. Optionally, the loss function may also depend on $\alpha(\cdot \mid X_P, \Sigma)$ for reference profile α (see NashPG).

The **training** is then an inductive process defining a sequence of weights $(\theta_k)_{k\geq 0}$. The first choice θ_0 is a random seed sampled from some initial distribution. Assuming we constructed $\theta_0, \dots, \theta_k$, we define the next iterate θ_{k+1} by: estimating $l(\theta)$ using the rollout profile θ_k, then performing some number of gradient descent steps on θ to minimize the estimated loss.

Assumptions. In the model above, we assume a few things that we justify here:

1. **No Q-head:** In many games of interests, the action space is too large to have a reasonnable Q-head as output of the model.
2. **No other-player value head:** With imperfect-information games, the state of the other player is not known. To output a value function for each player, one would need to $\sum_p |\mathcal{X}_p|$ outputs, which is typically too large to be practical. It's also not a quantity that we necessarily want the model to spend effort to learn: the expected returns over other players **without** conditioning on their private state is a much more relevant quantity to learn (see **??**).
3. **Synchronous rollout:** This is purely a theoretical assumption to simplify the exposition.

These assumptions shape the estimators and losses that are relevant for our context.

Primitives. For $0 \leq i \leq k$, we let $\rho_i^\theta = \frac{\pi^\theta(A|X_P,\Sigma)}{\pi^{\theta_i}(A|X_P,\Sigma)}$. We define **centered logits** as

$$z^\theta(a \mid X_P, \Sigma) := \log \pi^\theta(a \mid X_P, \Sigma) - \frac{1}{|\mathcal{E}_\Sigma|} \sum_{b \in \mathcal{E}_\Sigma} \log \pi^\theta(b \mid X_P, \Sigma) \tag{3.5}$$

Write $\widehat{A} = \widehat{Q}(X_P, \Sigma A) - V^{\theta_k}(X_P, \Sigma)$. If we are able to compute one, for $b \in \mathcal{E}_\Sigma$, write $\widehat{Q}(b) = \widehat{Q}(X_P, \Sigma b)$ an estimator of an all-action target. We also define a clipped value function $V^\theta_{\texttt{clip}} = V^{\theta_k} + \mathrm{clip}(V^\theta - V^{\theta_k}, -\varepsilon_V, \varepsilon_V)$. We then have the following classic losses (the notation $\mathcal{L}[\widehat{E}]$ means "the loss $\mathcal{L}$ depends on a choice of estimator $\widehat{E}$"):

$$\begin{aligned}
\mathcal{L}_{\texttt{PG}}[\widehat{A}] &:= -\rho_k^\theta \widehat{A}, \\
\mathcal{L}_{\texttt{clip}}[\widehat{A}] &:= -\min\Big\{\rho_k^\theta \widehat{A}, \mathrm{clip}\Big(\rho_k^\theta, 1-\varepsilon, 1+\varepsilon\Big)\widehat{A}\Big\} \\
\mathcal{L}_{\texttt{NeuRD}}[\widehat{Q}(-)] &:= -\sum_{b \in \mathcal{E}_\Sigma} z^\theta(b \mid X_P, \Sigma) \cdot \mathrm{clip}\Big(\widehat{Q}(b), -c_{\texttt{NeuRD}}, c_{\texttt{NeuRD}}\Big) \\
\mathcal{L}^{\texttt{sample}}_{\texttt{NeuRD}}[\widehat{Q}] &:= -\frac{z^\theta(A \mid X_P, \Sigma)}{\pi^{\theta_k}(A \mid X_P, \Sigma)} \cdot \mathrm{clip}\Big(\widehat{Q}, -c_{\texttt{NeuRD}}, c_{\texttt{NeuRD}}\Big) \\
\mathcal{L}_{\texttt{V}}[\widehat{V}] &:= c_V \Big(V^\theta(X_P, \Sigma) - \widehat{V}\Big)^2, \\
\mathcal{L}_{\texttt{V-clip}}[\widehat{V}] &:= c_V \max\Big\{\Big(V^\theta(X_P, \Sigma) - \widehat{V}\Big)^2, \Big(V^\theta_{\texttt{clip}}(X_P, \Sigma) - \widehat{V}\Big)^2\Big\}, \\
\mathcal{L}_{\texttt{H}} &:= -c_H \, \mathrm{H}\Big(\pi^\theta(\cdot \mid X_P, \Sigma)\Big), \\
\mathcal{L}_{\texttt{KL}} &:= c_{\texttt{KL}} \, \mathrm{KL}\Big(\pi^\theta(\cdot \mid X_P, \Sigma) \Big\| \pi^{\theta_{\mathrm{ref}(k)}}(\cdot \mid X_P, \Sigma)\Big) \\
\mathcal{L}^{\texttt{sample}}_{\texttt{KL}} &:= c_{\texttt{KL}}\Big(\rho_k^\theta \log(\rho_k^\theta) - \rho_k^\theta + 1\Big)
\end{aligned} \tag{3.6}$$

The displayed NeuRD losses clip raw Q-values and are therefore not invariant to additive reward shifts; they should be understood as deliberate non-invariant surrogates.

Relation to Theorem 2.1. The goal of this loss function is that its gradient resembles the gradient of the expected payoff $J_p(\pi_p^\theta, \mathrm{sg}[\pi_{-p}^\theta])$.

Practicality. In practice, it's essentially impossible to sample Σ in a non-biased way (one needs to first know the average length of the game). It's also impossible to create the estimators $\widehat{Q}$ and $\widehat{V}$ without seeing the sampled Σ as part of a full trajectory. These two constraints pushes us towards the approximations given in the following subsections.

3.2 Usual Implementation

We explain here the usual implementation of a training algorithm.

Algorithm 1: $\mathfrak{T} \leftarrow \texttt{SampleTerminal}(\pi, N)$

```
/* Terminal trajectory sampling used by ordinary sampled-private-state training.  */
```

Parameters: π, rollout profile; N, number of independent terminal rollouts
Output: $\mathfrak{T} = ((X_i, \mathsf{T}_i))_{i=1}^N$

for $i = 1, \dots, N$ **do**
  Sample $X_i \sim \mu$
  Set $\sigma \leftarrow \varnothing$
  while $\sigma \notin \mathcal{T}$ **do**
    $p \leftarrow o(\sigma)$
    Sample

$$E \sim \begin{cases} \pi(\cdot \mid X_{i,p}, \sigma) & \text{if } o(\sigma) \in \mathcal{P} \\ \chi(\cdot \mid X_i, \sigma) & \text{otherwise} \end{cases}$$

    Set $\sigma \leftarrow \sigma E$
  end
  Set $\mathsf{T}_i \leftarrow \sigma$
end
return $\mathfrak{T}$

Definition 3.1 (Empirical terminal-rollout decision law)**.** Fix a terminal rollout batch $\mathfrak{T} = ((X_i, \mathsf{T}_i))_{i=1}^N$ produced by algorithm 1. Define an index $I \in \{1, \dots, N\}$ by

$$\mathbb{P}(I = i \mid \mathfrak{T}) = \frac{|\mathsf{T}_i|_{\text{player}}}{\sum_{j=1}^N |\mathsf{T}_j|_{\text{player}}}. \tag{3.7}$$

Then let $\Sigma A \preccurlyeq \mathsf{T}_I$ be a uniform choice of a player decision and $P = o(\Sigma)$.

Do note that all convergence statements are restricted to the fixed finite game, fixed rollout profile, and the sampling schemes stated in their lemmas.

Lemma 3.1 (Convergence of the empirical terminal-rollout decision law)

Let $\widehat{\mathbb{P}}_N$ be the conditional law of (X_I, P, Σ, A) given $\mathfrak{T}$, and let $\mathbb{P}_\star$ be the ideal law of Notation 2.4. Then

$$\|\widehat{\mathbb{P}}_N - \mathbb{P}_\star\|_{\mathrm{TV}} \longrightarrow 0 \quad \text{almost surely}, \qquad \sqrt{N}(\widehat{\mathbb{P}}_N - \mathbb{P}_\star) \Longrightarrow \mathcal{N}(0, \Gamma)$$

for a covariance matrix Γ. Moreover, there are game-dependent constants $c, C > 0$ such that, for every $0 < \varepsilon < 1$,

$$\mathbb{P}\Big(\|\widehat{\mathbb{P}}_N - \mathbb{P}_\star\|_{\mathrm{TV}} > \varepsilon\Big) \le Ce^{-cN\varepsilon^2}.$$

Primitives. For $\lambda \in [0,1]$, $1 \le i \le N$, $\sigma a \preccurlyeq \mathsf{T}_i$ and $p = o(\sigma)$, we write

$$\widehat{Q}^\lambda(X_{i,p}, \sigma a) := \lambda^{|\mathsf{T}_i \setminus \sigma|_p} R_p(X_i, \mathsf{T}_i) + (1-\lambda) \sum_{\substack{\sigma \prec \rho \preccurlyeq \mathsf{T}_i \\ o(\rho)=p}} \lambda^{|\rho \setminus \sigma|_p - 1} V^{\theta_k}(X_{i,p}, \rho) \tag{3.8}$$

and $\widehat{A}^\lambda = \widehat{Q}^\lambda(X_{I,P}, \Sigma A) - V^{\theta_k}(X_{I,P}, \Sigma)$ and $\widehat{V}^\lambda = \widehat{Q}^\lambda(X_{I,P}, \Sigma A)$. Note that $\lambda = 1$ recovers the Monte Carlo return estimator (necessary in the absence of a value head).

Lemma 3.2 (Player-clock GAE)

The return of Equation 3.8 is the usual GAE lambda return for terminal-only rewards, $\gamma = 1$, and the successive decisions of player p as time steps.

For $|\mathcal{P}| = 2$, $\eta > 0$ and $(x, \tau) \in \mathcal{X} \times \mathcal{T}$, we define the **R-NaD modified return** as

$$R_p^{\mathtt{NaD}}(x, \tau) := R_p(x, \tau) + \eta \sum_{\substack{\sigma a \preccurlyeq \mathsf{T} \\ o(\sigma) \in \mathcal{P}}} (1 - 2\mathbb{1}_{o(\sigma)=p}) \log\left(\frac{\pi^{\theta_k}(a \mid x_{o(\sigma)}, \sigma)}{\pi^{\theta_{\mathrm{ref}(k)}}(a \mid x_{o(\sigma)}, \sigma)}\right) \tag{3.9}$$

Using this instead of R in the formula above, we get $\widehat{Q}^{\eta,\lambda}_{\mathtt{NaD}}$, $\widehat{A}^{\eta,\lambda}_{\mathtt{NaD}}$ and $\widehat{V}^{\eta,\lambda}_{\mathtt{NaD}}$. For $b \in \mathcal{E}_\Sigma$, we define the **R-NaD all-action return**

$$\begin{aligned}\widehat{Q}^{\eta,\lambda}_{\mathtt{NaD}}(b) = V^{\theta_k}(X_{I,P}, \Sigma) &- \eta \log\left(\frac{\pi^{\theta_k}(b \mid X_{I,P}, \Sigma)}{\pi^{\theta_{\mathrm{ref}(k)}}(b \mid X_{I,P}, \Sigma)}\right) \\ &+ \frac{\mathbf{1}_{\{b=A\}}}{\pi^{\theta_k}(b \mid X_{I,P}, \Sigma)} \left(\widehat{A}^{\eta,\lambda}_{\mathtt{NaD}} + \eta \log\left(\frac{\pi^{\theta_k}(b \mid X_{I,P}, \Sigma)}{\pi^{\theta_{\mathrm{ref}(k)}}(b \mid X_{I,P}, \Sigma)}\right)\right).\end{aligned} \tag{3.10}$$

Choices. Here are typical choices of losses used in training.

method	ref(k)	V^θ	minimized loss
`REINFORCE`	$\varnothing$	$\varnothing$ or fixed baseline	$\mathcal{L}_{\mathtt{PG}}[\widehat{A}^1]$
`Actor-critic`	$\varnothing$	V^θ	$\mathcal{L}_{\mathtt{PG}}[\widehat{A}^\lambda] + \mathcal{L}_{\mathtt{V}}[\widehat{V}^\lambda] + \mathcal{L}_{\mathtt{H}}$
`PPO`	$\varnothing$	V^θ	$\mathcal{L}_{\mathtt{clip}}[\widehat{A}^\lambda] + \mathcal{L}_{\mathtt{V}}[\widehat{V}^\lambda] + \mathcal{L}_{\mathtt{H}}$
`MMD-style PPO`	$\varnothing$	V^θ	$\mathcal{L}_{\mathtt{clip}}[\widehat{A}^\lambda] + \mathcal{L}_{\mathtt{V}}[\widehat{V}^\lambda] + \mathcal{L}_{\mathtt{H}} + \mathcal{L}_{\mathtt{KL}}^{\mathtt{sample}}$
`NashPG-PPO`	$K\lfloor k/K \rfloor$	V^θ	$\mathcal{L}_{\mathtt{clip}}[\widehat{A}^\lambda] + \mathcal{L}_{\mathtt{V}}[\widehat{V}^\lambda] + \mathcal{L}_{\mathtt{KL}}$
`RNaD-PPO`	$\mathrm{ref}(k)$	V^θ	$\mathcal{L}_{\mathtt{clip}}[\widehat{A}^{\mathtt{NaD},\lambda}] + \mathcal{L}_{\mathtt{V}}[\widehat{V}^{\mathtt{NaD},\lambda}]$
`R-NaD / DeepNash`	$\mathrm{ref}(k)$	V^θ	$\mathcal{L}_{\mathtt{NeuRD}}[\widehat{Q}^{\eta,\lambda}_{\mathtt{NaD}}(-)] + \mathcal{L}_{\mathtt{V}}[\widehat{V}^{\mathtt{NaD},\lambda}]$

The method names in this table denote synchronous loss surrogates, not exact reproductions or convergence claims. In `R-NaD`, it is not clear what function $\mathrm{ref}(k)$ to choose, as the original implementation is `IMPALA`-style rather than synchronous.

Implementation. Below is a pseudocode of the usual implementation.

Algorithm 2: $(\theta_k)_{k\geq 0} \leftarrow \texttt{SequentialPG}(\theta_0)$

```
/* Sequential rollout policy-gradient template corresponding to the procedure above.
   */
```

Parameters: θ_0, initial weights; θ, working copy of the next iterate
Hyperparameters: N, rollout count; B, minibatch size; chosen number of gradient-descent steps per iterate
Output: $(\theta_k)_{k\geq 0}$

for $k = 0, 1, 2, \ldots$ **do**
  Sample $(X_i, \mathsf{T}_i)_{i=1}^{N} \overset{\text{iid}}{\sim} (X, \mathsf{T}^{\pi^{\theta_k}})$ using $\texttt{SampleTerminal}(\pi^{\theta_k}, N)$
  for $i = 1, \ldots, N$ **do**
    Compute $R(X_i, \mathsf{T}_i)$ (optionally, using Equation 3.9)
    By backward propagation, for $\sigma a \preccurlyeq \mathsf{T}_i$ and $p = o(\sigma)$, compute $\widehat{Q}(X_{i,p}, \sigma a)$ using Equation 3.8
    Compute $\widehat{V}(X_{i,p}, \sigma)$
  end
  $\theta \leftarrow \theta_k$
  for *each gradient-descent step* **do**
    Sample $(I_b, \Sigma_b, A_b, P_b)_{b=1}^{B} \overset{\text{iid}}{\sim} (I, \Sigma, A, P)$ as in Definition 3.1
    Deduce $\widehat{A}_b$, $\widehat{Q}_b$, and $\widehat{V}_b$ at $(X_{I_b,P_b}, \Sigma_b, A_b)$
    Estimate

$$\widehat{l}(\theta) = \frac{1}{B}\sum_{b=1}^{B} \mathcal{L}\Big((\pi^{\theta}, V^{\theta}, \pi^{\theta_k}, V^{\theta_k})(X_{I_b,P_b}, \Sigma_b), A_b, \widehat{A}_b, \widehat{Q}_b, \widehat{V}_b\Big)$$

    Descend θ along $\nabla_\theta \widehat{l}(\theta)$
  end
  store $\theta_{k+1} = \theta$
end
return $(\theta_k)_{k\geq 0}$

Optionally, for some $\mathrm{ref}(k) < k$ reference index, during minibatches, we can evaluate $\pi^{\theta_{\mathrm{ref}(k)}}(\cdot \mid X_{I,P}, \Sigma)$ at the sampled (I, P, Σ) to modify the loss function (see NashPG).

3.3 Rao–Blackwellization of the Private State

Motivation. Game strategies often involve disguising one's private information. In our leading example, poker, good players must "bluff" (pretend to have good cards when they don't), "slowplay" (pretend to have bad cards when they don't) and more generally, they must "protect their action ranges", meaning that $\mathbb{P}(\text{private state of } p \mid \text{public information})$ should not be too revealing. When learning poker, players are taught to "balance their ranges" and think "what would I do if I had a different hand?". Motivated by this, it's a natural choice to design models so that it doesn't see his private state x_p but rather decides what to do with any potential private state x_p. In other words, we do a currying transformation and see the model as

$$\theta \longmapsto \Big(x_p \in \mathcal{X}_p \mapsto \pi^{\theta}(\cdot \mid x_p, \sigma) \in \Delta(\mathcal{E}_\sigma)\Big) \tag{3.11}$$

which means that a single model run now outputs a collection of distributions. The goal of Rao–Blackwellization is to exploit this architecture in the training algorithm to reduce the variance of the loss estimator induced by the private state X.

Algorithm 3: $\mathfrak{T}^{\mathsf{pre-c}} \leftarrow \texttt{SamplePreTerminal}(\pi, N)$

```
/* Public pre-terminal sampling for Rao-Blackwellized training.  */
```

Parameters: π, rollout profile; N, number of independent public rollouts
Output: $\mathfrak{T}^{\mathsf{pre-c}} = ((\mathsf{T}_i^{\mathsf{pre-c}}, \mathfrak{r}_i))_{i=1}^N$

for $i = 1, \dots, N$ **do**
  Sample $X_i \sim \mu$
  Set $\sigma \leftarrow \varnothing$ and $\mathfrak{r}_i \leftarrow (1)_{q \in \mathcal{P} \cup \{\mathsf{c}\}}$
  while $\sigma \notin \mathcal{T}^{\mathsf{pre-c}}$ **do**
    Set $q \leftarrow o(\sigma)$
    Sample

$$E \sim \begin{cases} \pi(\cdot \mid X_{i,q}, \sigma) & \text{if } q \in \mathcal{P} \\ \chi(\cdot \mid X_i, \sigma) & \text{otherwise} \end{cases}$$

    Set

$$\mathfrak{r}_{i,q} \leftarrow \mathfrak{r}_{i,q} \odot \begin{cases} \pi(E \mid -, \sigma) & \text{if } q \in \mathcal{P} \\ \chi(E \mid -, \sigma) & \text{otherwise} \end{cases}$$

    Normalize $\mathfrak{r}_{i,q}$ and set $\sigma \leftarrow \sigma E$
  end
  Set $\mathsf{T}_i^{\mathsf{pre-c}} \leftarrow \sigma$
end
return $\mathfrak{T}^{\mathsf{pre-c}}$

Definition 3.2 (Empirical pre-terminal Rao–Blackwell decision law)**.** Fix a pre-terminal rollout batch $\mathfrak{T}^{\mathsf{pre-c}} = ((\mathsf{T}_i^{\mathsf{pre-c}}, \mathfrak{r}_i))_{i=1}^N$ produced by algorithm 3. Define $I \in \{1, \dots, N\}$ by

$$\mathbb{P}(I = i \mid \mathfrak{T}^{\mathsf{pre-c}}) = \frac{|\mathsf{T}_i^{\mathsf{pre-c}}|_{\text{player}}}{\sum_{j=1}^N |\mathsf{T}_j^{\mathsf{pre-c}}|_{\text{player}}}. \tag{3.12}$$

Then let $\Sigma A \preccurlyeq \mathsf{T}_I^{\mathsf{pre-c}}$ be a uniform choice of a player decision and $P = o(\Sigma)$.

Lemma 3.3 (Convergence of the empirical pre-terminal decision law)

Let $\widehat{\mathbb{P}}_N^{\mathsf{pre-c}}$ be the conditional law of (P, Σ, A) given $\mathfrak{T}^{\mathsf{pre-c}}$, and let $\mathbb{P}_\star^{\mathsf{pre-c}}$ be the corresponding marginal of the ideal law of Notation 2.4. Then

$$\|\widehat{\mathbb{P}}_N^{\mathsf{pre-c}} - \mathbb{P}_\star^{\mathsf{pre-c}}\|_{\mathrm{TV}} \longrightarrow 0 \quad \text{almost surely,} \qquad \sqrt{N}(\widehat{\mathbb{P}}_N^{\mathsf{pre-c}} - \mathbb{P}_\star^{\mathsf{pre-c}}) \Longrightarrow \mathcal{N}(0, \Gamma_{\mathsf{pre-c}})$$

for a covariance matrix $\Gamma_{\mathsf{pre-c}}$. Moreover, there are game-dependent constants $c, C > 0$ such that, for every $0 < \varepsilon < 1$,

$$\mathbb{P}\Big(\|\widehat{\mathbb{P}}_N^{\mathsf{pre-c}} - \mathbb{P}_\star^{\mathsf{pre-c}}\|_{\mathrm{TV}} > \varepsilon\Big) \le C e^{-cN\varepsilon^2}.$$

Theory in the Ideal Setup. Essentially, we modify the expected value computations as

$$\begin{aligned} \widehat{Q} &= \mathbb{E}^\pi(R_p(x, \tau) \mid x_p, \sigma a) \\ &= \mathbb{E}^\pi(\mathbb{E}^\pi(R_p(x, \tau) \mid x_p, \tau^{\mathsf{pre-c}}) \mid x_p, \sigma a) \end{aligned} \tag{3.13}$$

$$\mathbb{E}(\mathcal{L}) = \mathbb{E}(\mathbb{E}^{\pi}(\mathcal{L}(x, \Sigma A) \mid \Sigma A)) \tag{3.14}$$

which reduces variance by the Law of Total Variance. For losses that depend nonlinearly on their targets, substituting these conditional means defines a new Rao–Blackwellized surrogate rather than the exact conditional expectation of the original loss; we expect the lower-variance targets to improve training nevertheless.

Primitives. For $\lambda \in [0,1]$, $1 \le i \le N$, $\sigma a \preccurlyeq \mathsf{T}_i$, $p = o(\sigma)$ and x_p compatible with T_i, we write

$$\widehat{Q}^{\lambda}(x_p, \sigma a) := \lambda^{|\mathsf{T}_i^{\mathsf{pre-c}} \setminus \sigma|_p} \mathbb{E}^{\pi^{\theta_k}}(R_p(x,\tau) \mid x_p, \mathsf{T}_i^{\mathsf{pre-c}} \preccurlyeq \tau) + (1-\lambda) \sum_{\substack{\sigma \prec \rho \preccurlyeq \mathsf{T}_i \\ o(\rho)=p}} \lambda^{|\rho \setminus \sigma|_p - 1} V^{\theta_k}(x_p, \rho) \tag{3.15}$$

and $\widehat{A}^{\lambda}(x_p) = \widehat{Q}^{\lambda}(x_p, \Sigma A) - V^{\theta_k}(x_p, \Sigma)$ and $\widehat{V}^{\lambda}(x_p) = \widehat{Q}^{\lambda}(x_p, \Sigma A)$.

Monte Carlo Methods. After experimentation, we found that a sequential importance sampler (SIS) resulted in the fastest convergence of the Monte Carlo estimates. Other methods such as rejection sampling or MCMC were found to be less efficient.

Algorithm 4: $(\widehat{\mathbb{P}}^{\mathfrak{r}}, \widehat{V}) \leftarrow \texttt{PreTerminalMC}(\mathfrak{r}, \tau^{\mathsf{pre-c}})$

```
/* Black-box Monte Carlo completion from public pre-terminal prefixes.  */
```

Parameters: $\mathfrak{r}$, reach cache; $\tau^{\mathsf{pre-c}}$, fixed pre-terminal trajectory
Hyperparameters: Monte Carlo budget
Output: $(\widehat{\mathbb{P}}^{\mathfrak{r}}, \widehat{V})$

1 Use a Monte Carlo simulation to sample $X \sim \mathbb{P}^{\mathfrak{r}}(\cdot \mid \tau^{\mathsf{pre-c}})$ with

$$\mathbb{P}^{\mathfrak{r}}(x \mid \tau^{\mathsf{pre-c}}) = \frac{\mu(x) \prod_{q \in \mathcal{P} \cup \{\mathsf{c}\}} r_q(\tau^{\mathsf{pre-c}} \mid x_q)}{\sum_{y \in \mathcal{X}} \mu(y) \prod_{q \in \mathcal{P} \cup \{\mathsf{c}\}} r_q(\tau^{\mathsf{pre-c}} \mid y_q)}$$

2 Conditionally on X and $\tau^{\mathsf{pre-c}}$, sample a terminal completion $\mathsf{T} \sim \mathbb{P}^{\pi}(\cdot \mid X, \tau^{\mathsf{pre-c}})$
3 Store statistical occurrence $(\widehat{\mathbb{P}}^{\mathfrak{r}}(x_p))_{\substack{p \in \mathcal{P} \\ x_p \in \mathcal{X}_p}}$ and statistical average

$$\left(\widehat{V}(x_p) \simeq \mathbb{E}(R_p(X, \mathsf{T}) \mid X_p = x_p, \tau^{\mathsf{pre-c}})\right)_{\substack{p \in \mathcal{P} \\ x_p \in \mathcal{X}_p}}$$

return $(\widehat{\mathbb{P}}^{\mathfrak{r}}, \widehat{V})$

Algorithm 5: $(\theta_k)_{k\geq 0} \leftarrow$ `SequentialRaoBlackwellPG`(θ_0)

```
/* Sequential rollout policy-gradient template corresponding to the procedure above.
   */
```

Parameters: θ_0, initial weights; θ, working copy of the next iterate
Hyperparameters: N, rollout count; B, minibatch size; chosen number of gradient-descent steps per iterate
Output: $(\theta_k)_{k\geq 0}$

for $k = 0, 1, 2, \ldots$ **do**
  Sample $(\mathsf{T}_i^{\mathsf{pre-c}}, \mathfrak{r}_i)_{i=1}^N \overset{\text{iid}}{\sim} \mathsf{T}^{\pi^{\theta_k}}$ using `SamplePreTerminal`(π^{θ_k}, N)
  For $1 \leq i \leq N$, $p \in \mathcal{P}$ and $x_p \in \mathcal{X}_p$, estimate

$$\widehat{\mathbb{P}}(x_p \mid \mathsf{T}_i^{\mathsf{pre-c}}) \simeq \mathbb{P}^{\pi}(x_p \mid \mathsf{T}_i^{\mathsf{pre-c}})$$
$$\widehat{V}_p(x_p, \mathsf{T}_i^{\mathsf{pre-c}}) \simeq \mathbb{E}^{\pi}(R_p(x,\tau) \mid x_p, \mathsf{T}_i^{\mathsf{pre-c}}) \tag{3.16}$$

  using `PreTerminalMC`$(\mathfrak{r}_i, \mathsf{T}_i^{\mathsf{pre-c}})$
  for $i = 1, \ldots, N$ **do**
    By backward propagation, for $\sigma a \preccurlyeq \mathsf{T}_i$, $p = o(\sigma)$ and $x_p \in \mathcal{X}_p$, compute $\widehat{Q}(x_p, \sigma a)$ using Equation 3.15
    Compute $\widehat{V}(x_p, \sigma)$
  end
  $\theta \leftarrow \theta_k$
  for *each gradient-descent step* **do**
    Sample $(I_b, \Sigma_b, A_b, P_b)_{b=1}^B \overset{\text{iid}}{\sim} (I, \Sigma, A, P)$ as in Definition 3.2
    Deduce $\widehat{A}_b(-)$, $\widehat{Q}_b(-)$, and $\widehat{V}_b(-)$ at (Σ_b, A_b)
    Estimate

$$\widehat{l}(\theta) = \frac{1}{B}\sum_{b=1}^{B}\sum_{x_{P_b}} \mathcal{L}\Big((\pi^{\theta}, V^{\theta}, \pi^{\theta_k}, V^{\theta_k})(x_{P_b}, \Sigma_b), A_b, \widehat{A}_b(x_{P_b}), \widehat{Q}_b(x_{P_b}), \widehat{V}_b(x_{P_b})\Big) \cdot \widehat{\mathbb{P}}(x_{P_b} \mid \mathsf{T}_{I_b}^{\mathsf{pre-c}})$$

    Descend θ along $\nabla_\theta \widehat{l}(\theta)$
  end
  store $\theta_{k+1} = \theta$
end
return $(\theta_k)_{k\geq 0}$

3.4 Variance-Guided Branching

Rao–Blackwellization integrates potential private states knowing a public terminal trajectory. It leaves on the table another substantial source of variance: a state σa might end in many different terminal trajectories. We suggest here to modify sequential rollout sampling into a Monte Carlo Tree Search (MCTS) style of sampling. This approach is particularly useful in games because typically there is only one possible initial public state, or a small space thereof (we build this into the PBG formalism where different initial public states are different games). For example, in our leading example of Texas Hold'em cash poker, the initial public state only contains the stack size of each player (and maybe antes, straddles or other house rules): this is a much smaller space than the space of all potential prompts for a coding model.

We further improve MCTS by suggesting a variance-guided branching strategy: we branch more on the nodes with high downstream reward variance.

Branched Rollouts. We enrich the rollout into a branching random walk on the public tree. The design constraint, encouraged by the Rao–Blackwellized pipeline, is that the enrichment must be *public*: any stored importance weight depending on a sampled hidden assignment would contaminate every posterior downstream (Warning 3.2).

Algorithm 6: $(\mathfrak{S}^{\beta,\pi}, \ell^{\beta,\pi}, N, \widetilde{N}, \mathfrak{r}) \leftarrow \texttt{SampleTree}(\pi, \beta; C, M_{\texttt{safe}})$

/* Sampling procedure for the rollout tree of definition 3.3. */

Parameters: π, rollout profile; $\beta : \mathcal{S} \setminus \mathcal{T} \to [1, \infty)$, intensity; optional frozen cost head C and budget $M_{\texttt{safe}}$, with $M_{\texttt{safe}} = \infty$ disabling the safeguard

Output: sampled tree $\mathfrak{S}^{\beta,\pi}$, label map $\ell^{\beta,\pi}$, executed and nominal child counts $(N, \widetilde{N})$, and node-specific reach caches $\mathfrak{r}$

Set $\mathfrak{S}^{\beta,\pi} \leftarrow \{\varnothing\}$, $\ell^{\beta,\pi}(\varnothing) \leftarrow \varnothing$, $\mathfrak{r}_{\varnothing} \leftarrow (1)_{q \in \mathcal{P} \cup \{\mathsf{c}\}}$, and expansion queue $\mathcal{F} \leftarrow \{\varnothing\}$

while $\mathcal{F} \neq \varnothing$ **do**

  Compute the next forward batch $\mathcal{B} \subseteq \mathcal{F}$ and maintain $R_{\mathcal{F}} = \sum_{\mathsf{s} \in \mathcal{F}} C(\ell(\mathsf{s}))$ incrementally (or $R_{\mathcal{F}} = 0$ if $M_{\texttt{safe}} = \infty$)

  for $\mathsf{s} \in \mathcal{B}$ **do**

    Set $\sigma \leftarrow \ell^{\beta,\pi}(\mathsf{s})$

    if σ *is pre-terminal* **then**

      Set $\mathfrak{E}_{\mathsf{s}} \leftarrow \varnothing$ and $(N(\mathsf{s}), \widetilde{N}(\mathsf{s})) \leftarrow (0, 0)$

    else

      Set $q \leftarrow o(\sigma)$

      Let $\widetilde{N}(\mathsf{s}) \sim \lfloor \beta(\sigma) \rfloor + \text{Bernoulli}\left(\beta(\sigma) - \lfloor \beta(\sigma) \rfloor\right)$, and set $N(\mathsf{s}) \leftarrow 1$ if $q \in \mathcal{P}$ and $|\mathfrak{S}^{\beta,\pi}| + R_{\mathcal{F}} > M_{\texttt{safe}}$, and $N(\mathsf{s}) \leftarrow \widetilde{N}(\mathsf{s})$ otherwise

      Set $\mathfrak{E}_{\mathsf{s}} \leftarrow \{\mathsf{a}_1, \ldots, \mathsf{a}_{N(\mathsf{s})}\}$ to be the set of children of s

      for $\mathsf{a} \in \mathfrak{E}_{\mathsf{s}}$, *independently* **do**

        Sample $X_{\mathsf{s},\mathsf{a}}$ from the posterior represented by the current reach cache,

$$\mathbb{P}^{\mathfrak{r}_{\mathsf{s}}}(x \mid \sigma) = \frac{\mu(x) \prod_{u \in \mathcal{P} \cup \{\mathsf{c}\}} r_{\mathsf{s},u}(\sigma \mid x_u)}{\sum_{y \in \mathcal{X}} \mu(y) \prod_{u \in \mathcal{P} \cup \{\mathsf{c}\}} r_{\mathsf{s},u}(\sigma \mid y_u)}$$

        Sample

$$E_{\mathsf{a}} \sim \begin{cases} \pi(\cdot \mid X_{\mathsf{s},\mathsf{a},q}, \sigma) & \text{if } q \in \mathcal{P}, \\ \chi(\cdot \mid X_{\mathsf{s},\mathsf{a}}, \sigma) & \text{otherwise} \end{cases}$$

        Set $\ell^{\beta,\pi}(\mathsf{sa}) \leftarrow \sigma E_{\mathsf{a}}$ and $\mathfrak{r}_{\mathsf{sa}} \leftarrow \mathfrak{r}_{\mathsf{s}}$

        Set

$$\mathfrak{r}_{\mathsf{sa},q} \leftarrow \mathfrak{r}_{\mathsf{sa},q} \odot \begin{cases} \pi(E_{\mathsf{a}} \mid -, \sigma) & \text{if } q \in \mathcal{P}, \\ \chi(E_{\mathsf{a}} \mid -, \sigma) & \text{otherwise} \end{cases}$$

        Normalize $\mathfrak{r}_{\mathsf{sa},q}$

      end

    end

  end

  Set $\mathcal{A}_{\mathcal{B}} \leftarrow \{\mathsf{sa} : \mathsf{s} \in \mathcal{B}, \mathsf{a} \in \mathfrak{E}_{\mathsf{s}}\}$, $\mathfrak{S}^{\beta,\pi} \leftarrow \mathfrak{S}^{\beta,\pi} \sqcup \mathcal{A}_{\mathcal{B}}$, $\mathcal{F} \leftarrow (\mathcal{F} \setminus \mathcal{B}) \sqcup \mathcal{A}_{\mathcal{B}}$, and update $R_{\mathcal{F}}$ by the corresponding removals and insertions

end

return $(\mathfrak{S}^{\beta,\pi}, \ell^{\beta,\pi}, N, \widetilde{N}, \mathfrak{r})$

Remark 3.1 (Practical Implementation). Note that practically speaking, to sample each outgoing

event E_{a}, we first sample a temporary private state x conditioned on σ (for example, in poker, we do this by rejection sampling), then we sample according to the chance rules and player strategies. Since $\beta \geq 1$, randomized rounding always gives at least one child; this guaranteed child is a *spine* which preserves every lineage. Every abstract sampled child is retained as a distinct tree node, even when two event labels E_{a} agree. Such nodes may share immutable public-state payloads, but never topology, descendants, random draws, or Monte Carlo completion samples; each outgoing edge has transition probability $1/N(\mathsf{s})$. At $\beta \equiv 1$ the tree is the set of nodes in a random trajectory $\mathsf{T} \sim \mathbb{P}^{\pi}(\cdot)$.

Definition 3.3 (Rollout tree)**.** Let $\beta : \mathcal{S} \setminus \mathcal{T} \to [1, \infty)$ be an intensity function and $(\mathfrak{S}^{\beta,\pi}, \ell^{\beta,\pi} : \mathfrak{S}^{\beta,\pi} \to \mathcal{S}, N^{\beta,\pi} : \mathfrak{S}^{\beta,\pi} \to \{0, 1, \ldots\}, \widetilde{N}^{\beta,\pi}, \mathfrak{r}) \leftarrow \texttt{SampleTree}(\pi, \beta; \bot, \infty)$ be a random labeled tree, seen as a Markov chain (with uniform transition probabilities), together with its auxiliary node-specific reach caches. Here $N^{\beta,\pi} = \widetilde{N}^{\beta,\pi}$. We have a natural probability distribution on $\mathcal{X} \times \mathcal{T}$ induced by first sampling a leaf of this Markov chain and then completing its pre-terminal public label:

$$\mathbb{P}^{\beta,\pi}(x, \tau \mid \mathfrak{S}^{\beta,\pi}) = \sum_{\ell(\mathsf{t}) = \tau^{\mathsf{pre-c}}} \left(\prod_{\mathsf{s} \prec \mathsf{t}} \frac{1}{N^{\beta,\pi}(\mathsf{s})} \right) \mathbb{P}^{\pi}(x, \tau \mid \tau^{\mathsf{pre-c}}). \tag{3.17}$$

From it, we can firstly define $P^{\beta,\pi} \in \mathcal{P}$ by

$$\mathbb{P}(P^{\beta,\pi} = p \mid \mathfrak{S}^{\beta,\pi}) = \frac{\mathbb{E}^{\beta,\pi}(|\tau|_p \mid \mathfrak{S}^{\beta,\pi})}{\mathbb{E}^{\beta,\pi}(|\tau|_{\text{player}} \mid \mathfrak{S}^{\beta,\pi})} \tag{3.18}$$

Secondly, we denote $\mathsf{S}^{\beta,\pi}$ for a random node sampled from the stationary distribution of the Markov chain defined on the sub-tree of $\mathfrak{S}^{\beta,\pi}$ on these nodes labeled by a game node owned by $P^{\beta,\pi}$, and we let $\Sigma^{\beta,\pi} = \ell(\mathsf{S}^{\beta,\pi})$ be its label. Thirdly, we let $\mathsf{A}^{\beta,\pi}$ be a uniform choice in $\mathfrak{E}_{\mathsf{S}^{\beta,\pi}}$ and let $A^{\beta,\pi}$ be its public event label, so $\ell(\mathsf{S}^{\beta,\pi}\mathsf{A}^{\beta,\pi}) = \Sigma^{\beta,\pi}A^{\beta,\pi}$. It's easy to see that the law of $\mathsf{S}^{\beta,\pi}$ is given by

$$\mathbb{P}(\mathsf{S}^{\beta,\pi} = \mathsf{s} \mid P^{\beta,\pi}, \mathfrak{S}^{\beta,\pi}) = \frac{\mathbb{1}_{o(\ell(\mathsf{s}))=P^{\beta,\pi}}}{\mathbb{E}^{\beta,\pi}(|\tau|_{P^{\beta,\pi}} \mid \mathfrak{S}^{\beta,\pi}) \prod_{\mathsf{r} \prec \mathsf{s}} N^{\beta,\pi}(\mathsf{r})} \tag{3.19}$$

which is indeed a probability distribution.

Lemma 3.4 (Single-tree convergence of the branched decision law)

Fix an intensity profile $\bar{\beta} : \mathcal{S} \setminus \mathcal{T} \to [1, \infty)$ and assume that every terminal trajectory contains at least one player node. For $K \in \mathbb{N}$, set $\beta_K := K\bar{\beta}$, sample a single rollout tree $\mathfrak{S}_K := \mathfrak{S}^{\beta_K,\pi}$ as in definition 3.3, and let

$$\widehat{\mathbb{P}}_K := \mathcal{L}\Big(P^{\beta_K,\pi}, \Sigma^{\beta_K,\pi}, A^{\beta_K,\pi} \Big| \mathfrak{S}_K\Big).$$

Let $\mathbb{P}^{\text{dec}}_{\star} := \mathcal{L}(P^{\pi}, \Sigma^{\pi}, A^{\pi})$ be the corresponding marginal of the ideal law in Notation 2.4. Then $|\mathfrak{S}_K| \to \infty$ and

$$\left\| \widehat{\mathbb{P}}_K - \mathbb{P}^{\text{dec}}_{\star} \right\|_{\text{TV}} \xrightarrow[K \to \infty]{\mathbb{P}} 0.$$

Lemma 3.5 (Terminal node distribution)

We have

$$\mathbb{E}(\mathbb{P}^{\beta,\pi}(\cdot \mid \mathfrak{S}^{\beta,\pi})) = \mathbb{P}^{\pi}(\cdot) \tag{3.20}$$

Lemma 3.6 (Branch independence)

Fix a sampled-tree node s *with public label* σ *and sampled outgoing-action set* $\mathfrak{E}_{\mathsf{s}}$*, and put* $m = N(\mathsf{s}) = |\mathfrak{E}_{\mathsf{s}}|$*. Conditional on* σ *and* m*, the rooted child branches*

$$\Big(\ell(\mathsf{sa}), \mathfrak{S}^{\beta,\pi}_{\mathsf{sa}\preccurlyeq}\Big)_{\mathsf{a}\in\mathfrak{E}_{\mathsf{s}}}$$

are mutually independent; in particular, any statistic of one child branch is independent of any statistic of the sibling forest, conditional on the public parent and the chosen child label.

Warning 3.2 (Don't sample a true hand!). One might consider sampling X first and then generating the sub-tree using the knowledge that X is the true private state. While this still satisfies Lemma 3.5 marginally, it violates Lemma 3.6. The sub-trees sampled by this process would be "self-correlated": for $\mathsf{a} \neq \mathsf{b} \in \mathfrak{E}_{\mathsf{s}}$, the sub-trees $\mathfrak{S}_{\mathsf{sa}\preccurlyeq}$ and $\mathfrak{S}_{\mathsf{sb}\preccurlyeq}$ share the same sampled private state and would not be independent. In particular, the sibling average used as the leave-one-out baseline for $\widehat{A}^{\lambda}$ would no longer be independent of the target branch after conditioning on the target action label.

Primitive Functions. Back to a synchronous rollout training, we fix weights $\theta_0, \ldots, \theta_k$. We let $\mathfrak{S}$, ℓ, P, S, Σ, A, A, T, $\mathsf{T}^{\mathsf{pre-c}}$ and X be as above for $\pi = \pi^{\theta_k}$. Throughout this paragraph, $\mathbb{E}^{\beta,\pi}$ is conditional on the realized sampled labeled tree $\mathfrak{S}$; conditions involving $\mathfrak{S}$ are displayed only where they clarify the sampled-node law. For a sampled-tree node s, write $\mathfrak{E}_{\mathsf{s}}$ for its sampled outgoing-action set, so that the children are sa for $\mathsf{a} \in \mathfrak{E}_{\mathsf{s}}$. For a player node $\mathsf{s} \in \mathfrak{S}$ with $\ell(\mathsf{s}) = \sigma$, a sampled action $\mathsf{a} \in \mathfrak{E}_{\mathsf{s}}$ with $\ell(\mathsf{sa}) = \sigma a_{\mathsf{a}}$, $p = o(\sigma)$ and $x_p \in \mathcal{X}_p$, we define the estimators

$$\begin{aligned}
\widehat{Q}^{\lambda}(x_p, \mathsf{sa}) &= \mathbb{E}^{\beta,\pi}\Big(\lambda^{|\tau\setminus\sigma|_p} R_p(x,\tau)\Big| x_p, \mathsf{sa}, \mathfrak{S}\Big) \\
&\quad + (1-\lambda)\mathbb{E}^{\beta,\pi}(|\tau\setminus\sigma|_p \mid x_p, \mathsf{sa}, \mathfrak{S})\mathbb{E}\Big(\lambda^{|\Sigma\setminus\sigma|_p - 1} V^{\theta_k}(x_p,\Sigma)\Big| \mathsf{sa} \preccurlyeq \mathsf{S}, P = p, \mathfrak{S}\Big) \\
\widehat{V}^{\lambda}(x_p, \mathsf{s}) &= \mathbb{E}^{\beta,\pi}\Big(\lambda^{|\tau\setminus\sigma|_p} R_p(x,\tau)\Big| x_p, \mathsf{s}, \mathfrak{S}\Big) \\
&\quad + (1-\lambda)\mathbb{E}^{\beta,\pi}(|\tau\setminus\sigma|_p \mid \mathsf{s}, \mathfrak{S})\mathbb{E}\Big(\lambda^{|\Sigma\setminus\sigma|_p - 1} V^{\theta_k}(x_p,\Sigma)\Big| \mathsf{s} \prec \mathsf{S}, P = p, \mathfrak{S}\Big) \\
\widehat{V}^{\lambda,\neg\mathsf{a}}(x_p, \mathsf{s}) &= \mathbb{E}^{\beta,\pi}\Big(\lambda^{|\tau\setminus\sigma|_p + 1} R_p(x,\tau)\Big| x_p, \mathsf{s}, \neg\mathsf{sa}, \mathfrak{S}\Big) \\
&\quad + (1-\lambda)\mathbb{E}^{\beta,\pi}(1 + |\tau\setminus\sigma|_p | x_p, \mathsf{s}, \neg\mathsf{sa}, \mathfrak{S})\mathbb{E}\Big(\lambda^{|\Sigma\setminus\sigma|_p} V^{\theta_k}(x_p,\Sigma)\Big| \mathsf{s} \preccurlyeq \mathsf{S}, \mathsf{sa} \not\preccurlyeq \mathsf{S}, P = p, \mathfrak{S}\Big) \quad \text{if } N(\\
\widehat{V}^{\lambda,\neg\mathsf{a}}(x_p, \mathsf{s}) &= V^{\theta_k}(x_p, \sigma) \quad \text{if } N(\mathsf{s}) = 1 \\
\widehat{A}^{\lambda}(x_p, \mathsf{sa}) &= \widehat{Q}^{\lambda}(x_p, \mathsf{sa}) - \widehat{V}^{\lambda,\neg\mathsf{a}}(x_p, \mathsf{s})
\end{aligned}$$

Lemma 3.7 (Unnormalised bucket moments)

All of the above estimators are unbiased when multiplied by $\mathbb{1}_{x_p}$*.*

For $N(\mathsf{s}) > 1$, Lemma 3.6 makes the sibling average entering $\widehat{V}^{\lambda,\neg\mathsf{a}}$ independent of the target branch after conditioning on the target action label, which is needed for the lemma above. All of these targets can be computed by backward propagation (see algorithm below).

For a sampled (P, S, Σ), a sampled action $\mathsf{a} \in \mathfrak{E}_{\mathsf{S}}$ labeled by Σa_{a}, and a private state $x_P \in \mathcal{X}_P$, we use the local aliases $\widehat{Q}(x_P, \mathsf{a}) := \widehat{Q}^{\lambda}(x_P, \mathsf{Sa})$, $\widehat{A}(x_P, \mathsf{a}) := \widehat{A}^{\lambda}(x_P, \mathsf{Sa})$ and $\widehat{V}(x_P) := \widehat{V}^{\lambda}(x_P, \mathsf{S})$.

Variance-Based Intensities using KKT. Ideally, we'd like to pick β so to reduce the variance of the gradient estimator. In practice, this is too noisy to be useful, so we use the following proxy. Firstly, we assume that the GAE horizon is $\lambda = 1$ for the variance calculation, so as to simplify notation: in practice this means that we are controlling the Monte Carlo source of variance in the estimators. Secondly, we define the vector Monte Carlo value estimator

$$\widehat{\mathbf{V}}(\mathsf{s}) = \mathbb{E}^{\beta,\pi}(\mathbf{R}(x,\tau)|\mathsf{s},\mathfrak{S})$$

By Cauchy-Schwarz, we can bound the error of the gradient estimator by the squared error $\mathrm{SqEr}(\mathbf{V} \mid \mathfrak{S}) = \mathbb{E}\Big(\|\widehat{\mathbf{V}}(\mathsf{S}) - \mathbb{E}(\mathbf{R} \mid \Sigma)\|_2^2 \mid \mathfrak{S}\Big)$. A reasonnable proxy is then to minimize the expected squarred error:

$$\begin{aligned}\mathbb{E}\Big(\mathbb{E}(|\tau|_{\text{player}} \mid \mathfrak{S})\,\mathrm{SqEr}(\widehat{\mathbf{V}} \mid \mathfrak{S})\Big) &= \mathbb{E}^{\pi}(|\tau|_{\text{player}})\mathbb{E}(\mathrm{trCov}(\widehat{\mathbf{V}} \mid \Sigma)) \\ &\simeq \sum_{\sigma\in\mathcal{S}\setminus\mathcal{T}} \mathbb{P}^{\pi}(\sigma \preccurlyeq \tau)\mathbf{U}(\sigma)\left(\sum_{\substack{\rho\preccurlyeq\sigma \\ o(\rho)\in\mathcal{P}}} \frac{1}{\prod_{\rho\preccurlyeq\alpha\preccurlyeq\sigma}\beta(\alpha)}\right)\end{aligned}$$

for $\mathbf{U}(\sigma) = \mathrm{trCov}(\mathbb{E}(\mathbf{R} \mid \sigma A) \mid \sigma)$, the portion of the variance coming from the next decision only, subject to controlling the expected size of the sampled tree:

$$\mathbb{E}(|\mathfrak{S}|) = 1 + \sum_{\sigma\in\mathcal{S}\setminus\mathcal{T}} \mathbb{P}^{\pi}(\sigma \preccurlyeq \tau)\prod_{\rho\preccurlyeq\sigma}\beta(\rho) \leq M$$

for a budget M.

We see this as a `KKT` fixed point problem in the player-node variables $(\beta(\sigma))_{o(\sigma)\in\mathcal{P}}$, keeping chance-node intensities fixed, with rescaled inverse Lagrange multiplier κ. Throughout this calculation, $\beta(\sigma) > 0$ is a deterministic continuous mean-field approximation to the randomized offspring count $N(\mathsf{s})$ at $\ell(\mathsf{s}) = \sigma$. Within this mean-field approximation, we compute the derivatives

$$\begin{aligned}\frac{\partial}{\partial(1/\beta(\sigma))}\Big(\mathbb{E}(\mathrm{trCov}(\widehat{\mathbf{V}} \mid \Sigma))\Big) &= \frac{\mathbb{P}^{\pi}(\sigma \preccurlyeq \tau)}{\mathbb{E}^{\pi}(|\tau|_{\text{player}})}\left(\sum_{\substack{\rho\preccurlyeq\sigma \\ o(\rho)\in\mathcal{P}}} \frac{1}{\prod_{\rho\preccurlyeq\alpha\prec\sigma}\beta(\alpha)}\right)\left(\sum_{\sigma\preccurlyeq\eta} \frac{\mathbf{U}(\eta)\mathbb{P}^{\pi}(\eta \preccurlyeq \tau \mid \sigma)}{\prod_{\sigma\prec\alpha\preccurlyeq\eta}\beta(\alpha)}\right) \\ &= \frac{\mathbb{P}^{\pi}(\sigma \preccurlyeq \tau)}{\mathbb{E}^{\pi}(|\tau|_{\text{player}})}\left(\sum_{\substack{\rho\preccurlyeq\sigma \\ o(\rho)\in\mathcal{P}}} \frac{1}{\prod_{\rho\preccurlyeq\alpha\prec\sigma}\beta(\alpha)}\right)\mathrm{trCov}(\widehat{\mathbf{V}} \mid \sigma, N(\mathsf{s}) = 1) \\ \frac{\partial}{\partial\beta(\sigma)}(\mathbb{E}(|\mathfrak{S}|)) &= \left(\mathbb{P}^{\pi}(\sigma \preccurlyeq \tau)\prod_{\rho\prec\sigma}\beta(\rho)\right)\mathbb{E}(|\mathfrak{S}_{\mathsf{s}_{\preccurlyeq}}| - 1 \mid \ell(\mathsf{s}) = \sigma, N(\mathsf{s}) = 1)\end{aligned}$$

This motivates us to define two new positive heads whose regression estimands are the arithmetic conditional means of the targets defined below:

1. **Trace-Variance Head:** $T^{\theta}(\sigma) \simeq \mathbb{E}(\widehat{T}(\mathsf{s}) \mid \ell(\mathsf{s}) = \sigma, N(\mathsf{s}) \geq 2)$, for which $T^{\theta}(\sigma) = \mathrm{sigmoid}(t^{\theta}(\sigma))$ is appropriate;
2. **Cost Head:** $C^{\theta}(\sigma) \simeq \mathbb{E}(\widehat{C}^1(\mathsf{s}) \mid \ell(\mathsf{s}) = \sigma)$, for which $C^{\theta}(\sigma) = \exp(c^{\theta}(\sigma))$ is appropriate.

Then the continuous KKT update is

$$\beta_k(\sigma) \leftarrow \mathrm{clip}\left(\sqrt{\kappa_k \frac{T^{\theta_k}(\sigma)}{C^{\theta_k}(\sigma)} \frac{\left(\sum_{\substack{\rho\preccurlyeq\sigma \\ o(\rho)\in\mathcal{P}}} \frac{1}{\prod_{\rho\preccurlyeq\alpha\prec\sigma}\beta_k(\alpha)}\right)}{\prod_{\rho\prec\sigma}\beta_k(\rho)}}, 1, \beta_{\max}\right) \tag{3.22}$$

for $o(\sigma) \in \mathcal{P}$, while chance-node intensities remain fixed.

Queue budget safeguard. Algorithm 6 stores the nominal rounded count $\widetilde{N}$ but executes $N = 1$ for one player-node forward batch whenever the one-child queue forecast exceeds $M_{\texttt{safe}}$. This is a pressure safeguard rather than a hard memory bound. The T and C estimands remain those of the nominal tree; algorithm 8 corrects them using $(N, \widetilde{N})$. For $0 < \gamma_\kappa, \eta_\kappa \leq 1$, let $\overline{M}_k$ be an exponential moving average of the counterfactual size $M_k^{\texttt{cf}}$ defined in algorithm 9. After each adaptive rollout, update

$$\overline{M}_k \leftarrow (1 - \gamma_\kappa)\overline{M}_{k-1} + \gamma_\kappa M_k^{\texttt{cf}} \tag{3.23}$$

$$\log \kappa_{k+1} \leftarrow \operatorname{clip}\left(\log \kappa_k + \eta_\kappa \log \frac{M}{\overline{M}_k}, \log \kappa_{\min}, \log \kappa_{\max}\right). \tag{3.24}$$

The controller evaluates the KKT update in log space. Randomized rounding satisfies $\mathbb{E}(\widetilde{N}(\mathsf{s}) \mid \ell(\mathsf{s}) = \sigma) = \beta(\sigma)$; the mean-field KKT ignores the corresponding reciprocal-moment correction. Practically speaking, we can't immediately use these heads in training before they are somewhat trained. So we warmup the training on a fixed intensity $\beta_{\texttt{fix}}$ before starting this iterative process. To train the trace-variance head, for every sampled node s labeled by σ with $N(\mathsf{s}) \geq 2$, first compute the executed-tree trace sample covariance

$$\widehat{T}^{\texttt{exec}}(\mathsf{s}) = \frac{1}{N(\mathsf{s}) - 1} \sum_{\mathsf{a} \in \mathfrak{E}_{\mathsf{s}}} \left\| \widehat{\mathbf{V}}(\mathsf{sa}) - \widehat{\mathbf{V}}(\mathsf{s}) \right\|_2^2 \tag{3.25}$$

and set it to $\perp$ when $N(\mathsf{s}) = 1$. Using the same GAE horizon $\lambda \in [0, 1]$ as above, $\widehat{C}^\lambda(\mathsf{s})$ estimates nominal per-child subtree size and $\widehat{C}^1$ is its Monte Carlo target. The counterfactual recurrences are given directly in algorithm 8; the T correction is first-order when the shared queue couples later sibling work. For a given sample, we let $\widehat{T} = \widehat{T}(\Sigma)$ and $\widehat{C}^\lambda = \widehat{C}^\lambda(\Sigma)$ and introduce the Gamma-deviance losses

$$\mathcal{L}_T[\widehat{T}] = \begin{cases} c_T\left(\dfrac{\widehat{T} + \varepsilon_T}{T^\theta(\Sigma) + \varepsilon_T} + \log \dfrac{T^\theta(\Sigma) + \varepsilon_T}{\widehat{T} + \varepsilon_T} - 1\right) & \text{if } N(\mathsf{S}) \geq 2, \\ 0 & \text{if } N(\mathsf{S}) = 1, \end{cases} \tag{3.26}$$

$$\mathcal{L}_C[\widehat{C}^\lambda] = c_C\left(\frac{\widehat{C}^\lambda}{C^\theta(\Sigma)} + \log \frac{C^\theta(\Sigma)}{\widehat{C}^\lambda} - 1\right). \tag{3.27}$$

Their expected optima are the arithmetic conditional means required by the KKT derivatives, while their relative scale makes very small and very large targets have comparable influence.

Algorithm 7: $(\widehat{\mathbb{P}}^{\mathfrak{r}}, \widehat{V}, \widehat{\mathbf{V}}) \leftarrow \texttt{PreTerminalMomentMC}(\mathfrak{r}, \tau^{\mathsf{pre-c}})$

/* Black-box Monte Carlo completion from public pre-terminal prefixes. */

Parameters: $\mathfrak{r}$, leaf-specific reach cache; $\tau^{\mathsf{pre-c}}$, fixed pre-terminal trajectory

Hyperparameters: Monte Carlo budget

Output: $(\widehat{\mathbb{P}}^{\mathfrak{r}}, \widehat{V}, \widehat{\mathbf{V}})$

Use a Monte Carlo simulation to sample $X \sim \mathbb{P}^{\mathfrak{r}}(\cdot \mid \tau^{\mathsf{pre-c}})$ with

$$\mathbb{P}^{\mathfrak{r}}(x \mid \tau^{\mathsf{pre-c}}) = \frac{\mu(x) \prod_{q \in \mathcal{P} \cup \{\mathsf{c}\}} r_q(\tau^{\mathsf{pre-c}} \mid x_q)}{\sum_{y \in \mathcal{X}} \mu(y) \prod_{q \in \mathcal{P} \cup \{\mathsf{c}\}} r_q(\tau^{\mathsf{pre-c}} \mid y_q)}$$

Conditionally on X and $\tau^{\mathsf{pre-c}}$, sample $\mathsf{T} \sim \mathbb{P}^{\pi}(\cdot \mid X, \tau^{\mathsf{pre-c}})$

Store statistical occurrence $\left(\widehat{\mathbb{P}}^{\mathfrak{r}}(x_p)\right)_{\substack{p \in \mathcal{P} \\ x_p \in \mathcal{X}_p}}$, statistical average

$$\left(\widehat{V}(x_p) \simeq \mathbb{E}(R_p(X, \mathsf{T}) \mid X_p = x_p)\right)_{\substack{p \in \mathcal{P} \\ x_p \in \mathcal{X}_p}}$$

Compute $\widehat{\mathbf{V}} \leftarrow \left(\widehat{\mathbb{P}}^{\mathfrak{r}}(-_p) \cdot \widehat{V}(-_p)\right)_{p \in \mathcal{P}}$

return $(\widehat{\mathbb{P}}^{\mathfrak{r}}, \widehat{V}, \widehat{\mathbf{V}})$

In the backward implementation, tuple operations are componentwise. For any tuple-valued child expression $F(\bullet)$, write

$$\mathrm{Avg}_{\mathsf{s}}[F(\bullet)] := \frac{1}{N(\mathsf{s})} \sum_{\mathsf{a} \in \mathfrak{E}_{\mathsf{s}}} F(\mathsf{sa})$$

and

$$\mathrm{Avg}_{\mathsf{s},\neg\mathsf{a}}[F(\bullet)] := \begin{cases} \dfrac{1}{N(\mathsf{s}) - 1} \displaystyle\sum_{\mathsf{b} \in \mathfrak{E}_{\mathsf{s}} \setminus \{\mathsf{a}\}} F(\mathsf{sb}) & \text{if } N(\mathsf{s}) > 1, \\ \bot & \text{if } N(\mathsf{s}) = 1. \end{cases}$$

For tuples z, h of the same shape, set

$$\mathrm{Mix}_{\lambda}(z, h) := \begin{cases} \lambda z + (1 - \lambda) h & \text{if } z \text{ is present}, \\ h & \text{if } z = \bot. \end{cases}$$

The notation $-_p$ denotes the full private-state vector indexed by $x_p \in \mathcal{X}_p$. For posterior mass w and posterior-weighted moment m, write

$$\mathrm{Cond}(m, w) := \begin{cases} m/w & \text{if } w > 0, \\ 0 & \text{if } w = 0, \\ \bot & \text{if } m = w = \bot. \end{cases}$$

In numerical implementations the denominator may be clamped to a small positive constant.

Algorithm 8: $\mathcal{O} \leftarrow \texttt{BranchedValue}(\mathfrak{S}, V, T, C, \widetilde{N}, \mathfrak{r})$

/* Backward Propagation of Value Estimators. */

Parameters: $\mathfrak{S}$, executed labeled tree with child counts N; V, value head; T, trace-variance head; C, cost head; $\widetilde{N}$, nominal child counts; $\mathfrak{r}$, node-specific reach caches

Hyperparameters: λ, GAE horizon

Output: target table $\mathcal{O}$

Initialize target table $\mathcal{O} \leftarrow \varnothing$

for *terminal* $\mathsf{t} \in \mathfrak{S}$ *labeled by pre-terminal* $\tau^{\mathsf{pre-c}}$ **do**

Monte Carlo estimation of

$$\begin{aligned} \widehat{\mathbb{P}}(x_p \mid \mathsf{t}) &\simeq \mathbb{P}^{\pi}(x_p \mid \tau^{\mathsf{pre-c}}) \\ \widehat{V}_p(x_p, \mathsf{t}) &\simeq \mathbb{E}^{\pi}(R_p(x,\tau) \mid \tau^{\mathsf{pre-c}}, x_p) \\ \widehat{\mathbf{V}}(\mathsf{t}) &\simeq \mathbb{E}^{\pi}(\mathbf{R}(x,\tau) \mid \tau^{\mathsf{pre-c}}) \end{aligned} \tag{3.28}$$

for $p \in \mathcal{P}$ and $x_p \in \mathcal{X}_p$ using $\texttt{PreTerminalMomentMC}(\mathfrak{r}_{\mathsf{t}}, \tau^{\mathsf{pre-c}})$

Set $\widehat{M}_p(-_p, \mathsf{t}) \leftarrow \widehat{\mathbb{P}}(-_p \mid \mathsf{t}) \widehat{V}_p(-_p, \mathsf{t})$ for every $p \in \mathcal{P}$

Set $(\widehat{C}^{\lambda}(\mathsf{t}), \Delta(\mathsf{t})) \leftarrow (0, 0)$

Store $\mathcal{O}[\mathsf{t}].\text{posterior} \leftarrow (\widehat{\mathbb{P}}(-_p \mid \mathsf{t}))_{p \in \mathcal{P}}$

end

for *node* $\mathsf{s} \in \mathfrak{S}$ *non-terminal, backward* **do**

Set $\sigma \leftarrow \ell(\mathsf{s})$ and $\overline{\Delta}(\mathsf{s}) \leftarrow \text{Avg}_{\mathsf{s}}[\Delta(\bullet)]$

Compute

$$\Big(\big(\widehat{\mathbb{P}}(-_p \mid \mathsf{s}), \widehat{M}_p(-_p, \mathsf{s})\big)_{p \in \mathcal{P}}, \widehat{\mathbf{V}}(\mathsf{s}), \widehat{C}^{\lambda}(\mathsf{s})\Big) \leftarrow \text{Avg}_{\mathsf{s}}\Big[\Big(\big(\widehat{\mathbb{P}}(-_p \mid \bullet), \widehat{M}_p(-_p, \bullet)\big)_{p \in \mathcal{P}}, \widehat{\mathbf{V}}(\bullet), 1 + \widetilde{N}(\mathsf{s}\bullet)\widehat{C}^{\lambda}($$

Store $\mathcal{O}[\mathsf{s}].\text{posterior} \leftarrow (\widehat{\mathbb{P}}(-_p \mid \mathsf{s}))_{p \in \mathcal{P}}$

if $o(\sigma) \in \mathcal{P}$ **then**

Set $\widehat{T}(\mathsf{s}) \leftarrow \max\{0, \widehat{T}^{\mathsf{exec}}(\mathsf{s}) - \overline{\Delta}(\mathsf{s})\}$ if $N(\mathsf{s}) \geq 2$ and to $\perp$ otherwise; set $T^*(\mathsf{s}) \leftarrow \widehat{T}(\mathsf{s})$ if present and to $T(\sigma)$ otherwise; set $\Delta(\mathsf{s}) \leftarrow (T^*(\mathsf{s}) + \overline{\Delta}(\mathsf{s}))/N(\mathsf{s}) - T^*(\mathsf{s})/\widetilde{N}(\mathsf{s})$

$\big(\widehat{Q}^{\lambda}(-_{o(\sigma)}, \mathsf{sa})\big)_{\mathsf{a} \in \mathfrak{E}_{\mathsf{s}}} \leftarrow \Big(\text{Cond}\big(\widehat{M}_{o(\sigma)}(-_{o(\sigma)}, \mathsf{sa}), \widehat{\mathbb{P}}(-_{o(\sigma)} \mid \mathsf{sa})\big)\Big)_{\mathsf{a} \in \mathfrak{E}_{\mathsf{s}}}$

$\big(\widehat{V}^{\lambda, \neg \mathsf{a}}(-_{o(\sigma)}, \mathsf{s})\big)_{\mathsf{a} \in \mathfrak{E}_{\mathsf{s}}} \leftarrow$ $\Big(\text{Mix}_{\lambda}\big(\text{Cond}\big(\text{Avg}_{\mathsf{s}, \neg \mathsf{a}}\big[\widehat{M}_{o(\sigma)}(-_{o(\sigma)}, \bullet)\big], \text{Avg}_{\mathsf{s}, \neg \mathsf{a}}\big[\widehat{\mathbb{P}}(-_{o(\sigma)} \mid \bullet)\big]\big), V(-_{o(\sigma)}, \sigma)\big)\Big)_{\mathsf{a} \in \mathfrak{E}_{\mathsf{s}}}$

$\big(\widehat{A}^{\lambda}(-_{o(\sigma)}, \mathsf{sa})\big)_{\mathsf{a} \in \mathfrak{E}_{\mathsf{s}}} \leftarrow \big(\widehat{Q}^{\lambda}(-_{o(\sigma)}, \mathsf{sa}) - \widehat{V}^{\lambda, \neg \mathsf{a}}(-_{o(\sigma)}, \mathsf{s})\big)_{\mathsf{a} \in \mathfrak{E}_{\mathsf{s}}}$

$\widehat{V}_{o(\sigma)}(-_{o(\sigma)}, \mathsf{s}) \leftarrow \text{Cond}(\widehat{M}_{o(\sigma)}(-_{o(\sigma)}, \mathsf{s}), \widehat{\mathbb{P}}(-_{o(\sigma)} \mid \mathsf{s}))$

Store current targets $\mathcal{O}[\mathsf{s}].\text{targets} \leftarrow \big((\widehat{A}^{\lambda}(-_{o(\sigma)}, \mathsf{sa}), \widehat{Q}^{\lambda}(-_{o(\sigma)}, \mathsf{sa}))_{\mathsf{a} \in \mathfrak{E}_{\mathsf{s}}}, \widehat{V}_{o(\sigma)}(-_{o(\sigma)}, \mathsf{s}), \widehat{T}(\mathsf{s}), \widehat{C}^{\lambda}(\mathsf{s})\big)$

Overwrite with the bootstrapped private-state moment propagated to ancestors:

$$\widehat{M}_{o(\sigma)}(-_{o(\sigma)}, \mathsf{s}) \leftarrow \text{Mix}_{\lambda}\big(\widehat{M}_{o(\sigma)}(-_{o(\sigma)}, \mathsf{s}), \widehat{\mathbb{P}}(-_{o(\sigma)} \mid \mathsf{s}) V(-_{o(\sigma)}, \sigma)\big) \tag{3.30}$$

$$\widehat{C}^{\lambda}(\mathsf{s}) \leftarrow \text{Mix}_{\lambda}\big(\widehat{C}^{\lambda}(\mathsf{s}), C(\sigma)\big) \tag{3.31}$$

end

else

Set $\Delta(\mathsf{s}) \leftarrow \overline{\Delta}(\mathsf{s})/N(\mathsf{s})$

end

end

return *target table* $\mathcal{O}$

Note that returning $\widehat{Q}$ is only necessary for the deferred `NeuRD` research extension. The production PPO path returns $\widehat{A}$ and does not store $\widehat{Q}$ by default.

Algorithm 9: $(\theta_k)_{k\geq 0} \leftarrow$ `BranchedRaoBlackwellPG`(θ_0)

/* Tree-valued generalization of the sequential Rao-Blackwellized template; $\beta \equiv 1$ recovers it exactly. */

/* The implementation exposes fixed and post-warmup dynamic intensity modes; both train actor-type V and public T/C heads with PPO advantage targets. */

Parameters: θ_0, initial weights; θ, working copy of the next iterate

Hyperparameters: B, minibatch size; λ; $\beta_{\max}$; $M_{\texttt{safe}}$

Hyperparameters: $\beta_{\texttt{fix}}$, $k_{\texttt{warm}}$, M, κ_0, $\kappa_{\min}$, $\kappa_{\max}$, γ_κ, η_κ; chosen number of gradient-descent steps per iterate

Output: $(\theta_k)_{k\geq 0}$

Set $\overline{M}_{-1} \leftarrow M$

for $k = 0, 1, 2, \dots$ **do**

  Define $\beta_k \leftarrow \beta_{\texttt{fix}}$ if $k < k_{\texttt{warm}}$; otherwise define it from the frozen public heads T^{θ_k} and C^{θ_k}, the current inverse multiplier κ_k, and the KKT rule above

  Sample $(\mathfrak{S}, \ell, N, \widetilde{N}, \mathfrak{r}) \leftarrow \texttt{SampleTree}(\pi^{\theta_k}, \beta_k; C^{\theta_k}, M_{\texttt{safe}})$

  Set $M_k^{\texttt{cf}} \leftarrow |\mathfrak{S}| + \sum_{\mathsf{s}\in\mathfrak{S}} (\widetilde{N}(\mathsf{s}) - N(\mathsf{s})) C^{\theta_k}(\ell(\mathsf{s}))$. If $k \geq k_{\texttt{warm}}$, set $\overline{M}_k \leftarrow (1-\gamma_\kappa)\overline{M}_{k-1} + \gamma_\kappa M_k^{\texttt{cf}}$ and update $\log \kappa_{k+1}$ by the controller above; otherwise set $(\overline{M}_k, \kappa_{k+1}) \leftarrow (\overline{M}_{k-1}, \kappa_k)$

  Compute target table $\mathcal{O} \leftarrow \texttt{BranchedValue}(\mathfrak{S}, V^{\theta_k}, T^{\theta_k}, C^{\theta_k}, \widetilde{N}, \mathfrak{r})$

  for *each gradient-descent step* **do**

    Sample $(P_b, \mathsf{S}_b)_{b=1}^B \overset{\text{iid}}{\sim} (P, \mathsf{S})$ and set $\Sigma_b = \ell(\mathsf{S}_b)$

    Deduce $\widehat{A}_b^\lambda(x_{P_b}, \mathsf{a})$, $\widehat{Q}_b^\lambda(x_{P_b}, \mathsf{a})$, $\widehat{V}_b(x_{P_b})$, $\widehat{T}_b$ and $\widehat{C}_b^\lambda$ from $\mathcal{O}[\mathsf{S}_b]$.targets, and deduce $\widehat{\mathbb{P}}(x_{P_b} \mid \mathsf{S}_b)$ and $\widehat{\mathbb{P}}(x_{P_b} \mid \mathsf{S}_b\mathsf{a})$ from the corresponding posterior fields

    Estimate

$$\begin{aligned}\widehat{l}(\theta) = \frac{1}{B}\sum_{b=1}^{B} \sum_{\substack{x_{P_b} \\ \mathsf{a}\in\mathfrak{C}_{\mathsf{S}_b}}} \frac{1}{N(\mathsf{S}_b)} \mathcal{L}^\pi\Big((\pi^\theta, V^\theta, \pi^{\theta_k}, V^{\theta_k})(x_{P_b}, \Sigma_b), a_{\mathsf{a}}, \widehat{A}_b^\lambda(x_{P_b}, \mathsf{a}), \widehat{Q}_b^\lambda(x_{P_b}, \mathsf{a})\Big) \widehat{\mathbb{P}}(x_{P_b} \mid \mathsf{S}_b\mathsf{a}) \\ + \frac{1}{B}\sum_{b=1}^{B} \sum_{x_{P_b}} \mathcal{L}^V\Big((V^\theta, V^{\theta_k})(x_{P_b}, \Sigma_b), \widehat{V}_b(x_{P_b})\Big) \widehat{\mathbb{P}}(x_{P_b} \mid \mathsf{S}_b) \\ + \frac{1}{B}\sum_{b=1}^{B} \mathcal{L}^{T,C}\Big((T^\theta, C^\theta)(\Sigma_b), \widehat{T}_b, \widehat{C}_b^\lambda\Big)\end{aligned} \tag{3.32}$$

    Descend θ along $\nabla_\theta \widehat{l}(\theta)$

  end

  Store $\theta_{k+1} \leftarrow \theta$

end

return $(\theta_k)_{k\geq 0}$

4 Loss Estimator Variance

4.1 Sequential Rao–Blackwellization

Freeze $\pi = \pi^{\theta_k}$, evaluate derivatives at $\theta = \theta_k$, and take $\lambda = 1$ with loss $\mathcal{L}_{\mathtt{PG}} + \mathcal{L}_{\mathtt{V}}$. For a player decision $\nu = (\sigma, a)$, put

$$\Phi_k(x, \tau; \nu) := -\Big(R_{o(\sigma)}(x, \tau) - V^{\theta_k}(x_{o(\sigma)}, \sigma)\Big)\Big[\nabla_\theta \log \pi^\theta(a \mid x_{o(\sigma)}, \sigma) + 2c_V \nabla_\theta V^\theta(x_{o(\sigma)}, \sigma)\Big]_{\theta=\theta_k}.$$

Thus conditioning Φ_k on the public pre-terminal trajectory gives the exact Rao–Blackwellized gradient contribution. Throughout this subsection, $\mathrm{Var}(Y) := \mathrm{trCov}(Y)$.

Let $\mathfrak{T}^{\mathsf{pre-c}} = ((\mathsf{T}_i^{\mathsf{pre-c}}, \mathfrak{r}_i))_{i=1}^N$ be as in algorithm 3, let $D_i := |\mathsf{T}_i^{\mathsf{pre-c}}|_{\text{player}}$, $L := \sum_i D_i$, and $\ell := L/N$. Given the rollout batch, sample $((I_b, P_b, \Sigma_b, A_b))_{b=1}^B$ from definition 3.2. For every rollout i, use n conditionally iid completions, with completion j shared by all sampled decisions for which $I_b = i$. The scaled gradient estimator is

$$\widehat{g}_{N,B,n} := \frac{\ell}{nB} \sum_{j=1}^{n} \sum_{b=1}^{B} \Phi_k(X_{I_b,j}, \mathsf{T}_{I_b,j}; \Sigma_b, A_b).$$

The factor ℓ cancels the random denominator L in the empirical decision law, so $\widehat{g}_{N,B,n}$ is unbiased for the expected trajectory-sum gradient.

For a generic public pre-terminal trajectory, write $D := |\mathsf{T}^{\mathsf{pre-c}}|_{\text{player}}$ and $\ell_\star := \mathbb{E}D$. For every player decision $\nu \preccurlyeq \mathsf{T}^{\mathsf{pre-c}}$, define

$$\mu_\nu := \mathbb{E}^\pi(\Phi_k(X, \mathsf{T}; \nu) \mid \mathsf{T}^{\mathsf{pre-c}}), \qquad G := \sum_{\nu \preccurlyeq \mathsf{T}^{\mathsf{pre-c}}} \mu_\nu, \qquad Q := \sum_{\nu \preccurlyeq \mathsf{T}^{\mathsf{pre-c}}} \|\mu_\nu\|_2^2,$$

and $\xi_\nu := \Phi_k(X, \mathsf{T}; \nu) - \mu_\nu$. The constants governing rollout and minibatch variance are

$$\begin{aligned} A_{\text{rollout}} &:= \mathrm{Var}(G), \\ A_{\text{MB}}^{\neq} &:= \ell_\star \mathbb{E}Q - \|\mathbb{E}G\|_2^2, \\ A_{\text{MB}}^{=} &:= \mathbb{E}(DQ - \|G\|_2^2), \end{aligned} \tag{4.1}$$

while the completion constants are

$$\begin{aligned} C_{\text{MC}}^{\neq} &:= \ell_\star \mathbb{E} \sum_{\nu \preccurlyeq \mathsf{T}^{\mathsf{pre-c}}} \mathbb{E}^\pi(\|\xi_\nu\|_2^2 \mid \mathsf{T}^{\mathsf{pre-c}}), \\ C_{\text{MC}}^{=} &:= \mathbb{E}\left[D \sum_{\nu \preccurlyeq \mathsf{T}^{\mathsf{pre-c}}} \mathbb{E}^\pi(\|\xi_\nu\|_2^2 \mid \mathsf{T}^{\mathsf{pre-c}})\right], \\ C_{\text{MC}}^{\text{pair}} &:= \mathbb{E}\left[\mathbb{E}^\pi\left(\left\|\sum_{\nu \preccurlyeq \mathsf{T}^{\mathsf{pre-c}}} \xi_\nu\right\|_2^2 \middle| \mathsf{T}^{\mathsf{pre-c}}\right)\right]. \end{aligned} \tag{4.2}$$

Here A_{rollout} is public-trajectory variance; the superscripts $\neq$ and $=$ distinguish different-rollout and same-rollout terms in the second-moment expansion; and $C_{\text{MC}}^{\text{pair}}$ contains the covariance between decisions sharing one completion.

Successive applications of the law of total variance give the exact decomposition

$$\begin{aligned} \mathrm{Var}(\widehat{g}_{N,B,n}) &= T_{\text{rollout}}(N) + T_{\text{minibatch}}(N, B) + T_{\text{montecarlo}}(N, B, n), \\ T_{\text{rollout}}(N) &= \frac{A_{\text{rollout}}}{N}, \\ T_{\text{minibatch}}(N, B) &= \frac{A_{\text{MB}}^{\neq}}{B} + \frac{A_{\text{MB}}^{=} - A_{\text{MB}}^{\neq}}{BN}, \\ T_{\text{montecarlo}}(N, B, n) &= \frac{1}{n}\left(\frac{C_{\text{MC}}^{\neq}}{B} + \frac{C_{\text{MC}}^{\text{pair}}}{N} + \frac{C_{\text{MC}}^{=} - C_{\text{MC}}^{\neq} - C_{\text{MC}}^{\text{pair}}}{BN}\right). \end{aligned} \tag{4.3}$$

The $C_{\mathrm{MC}}^{\mathrm{pair}}/(Nn)$ term is the variance floor caused by sharing one completion across all selected decisions on a rollout; increasing B alone cannot remove it. All six constants are independent of (N, B, n).

5 Poker: a Perfect Example

We focus on configurable equal-stack Texas Hold'em cash poker: $P \in \{2, \dots, 6\}$ players, $S \in [8, 128]$ big blinds per player in half-big-blind increments, and the highest $R \in \{4, \dots, 13\}$ ranks with four suits. Let $D = 4R$ and require $2P + 5 < D$. There is no ante, straddle, or rake, and all legal actions in the fixed chip denomination are allowed. Below are a few core variables:

$$
\begin{aligned}
|\mathcal{P}| &= P \\
\mathcal{P} &= \{0, \dots, P-1\} \\
\mathcal{D}_R &= \{52 - D, \dots, 51\} \\
\mathcal{X}_p = \mathcal{H}_R &= \binom{\mathcal{D}_R}{2} \\
\mu &= \mathsf{Unif}(\{x \in \mathcal{X} \mid x_p \cap x_q = \varnothing \text{ for } p \neq q\}) \\
\mathsf{B}(\sigma) &\in \binom{\mathcal{D}_R}{0} \text{ or } \binom{\mathcal{D}_R}{3} \text{ or } \binom{\mathcal{D}_R}{4} \text{ or } \binom{\mathcal{D}_R}{5} \\
\mathsf{C}(x, \sigma) &= \left(\bigcup_p x_p\right) \cup \mathsf{B}(\sigma) \\
\chi(\cdot \mid x, \sigma) = \mathsf{Unif}\left(\binom{\mathcal{D}_R \setminus \mathsf{C}(x,\sigma)}{k(\sigma)}\right), &\qquad k(\sigma) = \begin{cases} 3 & \text{at the flop,} \\ 1 & \text{at the turn or river.} \end{cases}
\end{aligned}
$$

Several specificities impact our implementation:

1. We feed the model **redundant** information by feeding it both the public state σ and the Bayesian-updated ranges $\mathfrak{r}$.

2. The game of poker has a strong **preflop gauge symmetry**: any suit permutation should leave the strategy invariant. Instead of $\binom{4R}{2}$ possible private player states, there are R^2 classes (1326 and 169 respectively for the full deck). We exploit this by having a separate model for preflop and postflop, making it impossible for the model to not pick actions invariants under this transformation preflop. Optionally, we can also exploit this symmetry in the Monte Carlo estimators (see below) by storing the 4! suit permutations of each sampled private state and board into the accumulated statistics (this can be considered if possible with minimal wrap divergence).

3. The **postflop gauge symmetry** is harder to handle as it acts on the board which is an input. Optionally, we can add a loss function term that: samples a random suit permutation, applies it to the model inputs, and penalizes the difference between the re-permutated outputs and the original outputs. This adds one forward pass per sample, which shouldn't be too expensive compared to the cost of a backward pass.

4. In **postflop** states, to control the size of the input, we actually only feed the ranges of the **live** players (meaning those who have not folded yet and are thus still eligible to win part of the pot). We feed the model information about folded players by exploiting the very architecture of attention heads (see subsection to come on cluster expansions and Möbius expansions). This adds a computational step which can be seen as the first step of the model (and can be launched in the same batches as the model itself).

5. The model is a multi-headed attention transformer model, with one token per pair (p, h) (for p only a live player postlfop), one token for each piece of public information, and one taken per **action mode**. The tokens corresponding to action a and pair $(p_{\text{acting},h})$ are used to output propensity for a to be played with hand h.

6. As a consequence, we need to either:

   (a) **Memory Pressure:** during rollout, store the live ranges of each rolled-out state, so they can be fed to the minibatch samples as inputs

   (b) **Redundant Computations:** during minibatches, recompute the live ranges of each samples state Σ by quickly walking back the tree from Σ to $\varnothing$, and doing $\odot$-Bayesian updates along the way.

7. Three operations are already implemented in `CUDA` and `C++` in a separate repository:

   (a) The **folded statistics** to be fused with the model as its first step. This is **only** used: right after a player folded (otherwise can be stored and passed down the trajectory), and postlfop (preflop we can afford feeding the model all ranges).

   (b) A **rejection sampler** to be used exclusively for the branching rollout, and to be fused with the model as its last step,

   (c) A **Monte Carlo estimator** of equity and posteriors, used by both Rao–Blackwellized variants after rollouts are completed and before backpropagation to compute $\widehat{Q}$, $\widehat{V}$, ...

   Folded statistics might be more efficiently computed by the CPU considering that they are only randomly necessary. The other two operations should really be GPU-based.

8. We implement a synchronous rollout that is heavily computed by GPUs (typically a node of 2-4). We maintain a queue of states to be rolled out, and we roll them out in batches (two types of batches: one for preflop, one for postflop).

9. **Research loss design:** the full design combines clipped `PPO` with an optional `KL` term to an older reference index $\mathrm{ref}(k)$, an entropy term, and losses for each proposed value head:

$$\mathcal{L}_{\texttt{clip}}[\widehat{A}^{\lambda}] + \mathcal{L}_{\texttt{H}} + \mathcal{L}_{\texttt{KL}} + \mathcal{L}_{\texttt{V}}[\widehat{V}^{\lambda}] + \mathcal{L}^{T,C}[\widehat{T}, \widehat{C}^{\lambda}]$$

   The implementation companion exposes explicit `fixed` and `dynamic` branching modes. Both use the clipped policy, actor-type value, entropy, optional exact legal-action reference-policy KL, and public T/C losses from the first update. The dynamic mode follows $\beta_{\texttt{fix}}$ during warmup and only afterward lets the KKT controller consume T/C for rollout control. `NeuRD` and stored all-action Q targets remain deferred research extensions.

10. Ideally, the implemtation should be clearly modular: it will be used for poker, but must be easily adaptable to other PBGs.